Pytest Assertions Through a Unit Enrolment Project

Software Technology 1 - companion practice resource

Many assertion forms, one OOP project, independently verified tests

1. The project and how to use it

This PDF contains everything needed to build and test a small OOP unit enrolment application: the complete application code, folder layout, dependency line, first test, fixtures and a larger assertion catalogue. Create the files yourself in a separate practice folder by following the labelled code sections. No downloadable Python project is required.

Component	Responsibility
Student	Immutable ID/name value object. Both trimmed fields must be non-blank text. Equality compares their values.
Unit	Owns a roster and marks. Capacity is a positive integer; the roster preserves enrolment order.
enrol(student)	True for a new ID, False for a duplicate ID. Keeps the original object. A new ID at capacity raises ValueError.
record_mark(id, mark)	Enrolled students only. Finite numeric marks 0–100 inclusive; no booleans. A later mark replaces an earlier mark.
Public observations	Roster tuple snapshot, independent marks dictionary, remaining places, lookup, mean and summary. No marks means a None mean.
display_unit	Prints the documented summary followed by one newline.

These deliberately small practice rules are not a university's published enrolment policy. There is no timetable, prerequisite, authentication or persistence subsystem. The average uses recorded marks only, excluding ungraded students.

Work through sections 2–7 to construct the application and write your first test. Section 8 explains how to assemble the complete test file using the twelve groups in sections 9–20. The complete catalogue was verified as **51 passed**, from 42 test functions including parametrised cases, using Python 3.13.0 and pytest 8.3.5. A passing test is evidence about its scenario and assertions, not proof of complete software correctness.

pytest tests use ordinary Python `assert` statements. `pytest.raises` checks expected exceptions; `pytest.approx` supports justified numerical tolerances. Each assertion must describe a promised observation.

2. Create the practice folder and files

Create a folder named `unit_enrolment_practice` in your editor, outside the assessed SmartCare repository. Inside it, create a `tests` folder. The finished layout is:

```
unit_enrolment_practice/
├── enrolment.py
├── requirements-dev.txt
└── tests/
    ├── conftest.py
    └── test_assertions.py
```

File you create	Content supplied in this PDF
enrolment.py	Append the four code parts in sections 3–6, in order, into one file.
requirements-dev.txt	The single dependency line below.
tests/test_assertions.py	First version in section 7. Later replace it with section 8's imports and every example in sections 9–20.
tests/conftest.py	Complete fixture setup in section 8; not needed for the first local-setup test.

Put exactly this line in `requirements-dev.txt`:

```
pytest==8.3.5
```

Select the teaching team's confirmed Python interpreter/environment. Open a terminal in the practice folder. If the terminal starts in its parent, enter it first:

```
cd unit_enrolment_practice
python --version
python -m pip install -r requirements-dev.txt
```

All later pytest commands run from this folder, where `enrolment.py` lives. Do not run them from `tests/`: the application import and relative file paths depend on the working directory.

Copy plain code only, retaining spaces and line breaks. PDF code blocks are listings, not separate downloads. If copying from a PDF changes indentation or quotation marks, correct them in the editor before running Python. Do not include headings or page furniture in a source file.

Application code defines behaviour; test code states the independently expected result. Keep these roles in separate files. You do not need an `__init__.py` file or a packaging framework for this small layout with `python -m pytest` from its root.

3. Application code, part 1: Student

Start the empty `enrolment.py` file with the complete code below. `Student` is a value object: it describes one student's ID and name. The dataclass decorator generates construction and field-based equality, so two separate Student instances with equal fields compare equal.

```
"""Independent teaching project: an in-memory unit roster and marks.

This deliberately small model has no university integrations or SmartCare code.
"""
from dataclasses import dataclass
from math import isfinite


@dataclass(frozen=True)
class Student:
    """Immutable value object; equality compares both declared fields."""

    student_id: str
    name: str

    def __post_init__(self):
        for field in ("student_id", "name"):
            value = getattr(self, field)
            if not isinstance(value, str):
                raise TypeError(f"{field} must be text")
            if not value.strip():
                raise ValueError(f"{field} must not be blank")
            object.__setattr__(self, field, value.strip())
```

`frozen=True` prevents normal reassignment of fields after construction. During initial validation, `object.__setattr__` stores the trimmed values. These are Python class mechanics, not pytest features. The double-underscore `__post_init__` method is called after the generated constructor has assigned the fields.

The checks reject non-text and blank inputs. A type hint documents the intended value but does not validate it at runtime; the explicit checks do. Tests will observe public values, object identity and documented exceptions.

Save the file, then append the Unit class in the next section. Do not create a second application module for the later parts.

4. Application code, part 2: Unit construction

Append this code below Student in the same `enrolment.py` file. `Unit` owns its roster and marks. Its constructor creates fresh dictionaries for each instance, so two units do not share a roster accidentally.

```
class Unit:
    """Own a roster in enrolment order and zero or one mark per student.

    Capacity is a positive integer. Re-enrolling a known ID returns False,
    retains the original Student and does not consume another place, even
    when full. A new ID at capacity raises ValueError without changing state.
    Marks are finite numbers from 0 to 100 inclusive; booleans are excluded.
    Recording a later mark replaces that student's earlier mark.
    """

    def __init__(self, code: str, capacity: int):
        if not isinstance(code, str):
            raise TypeError("code must be text")
        if not code.strip():
            raise ValueError("code must not be blank")
        if isinstance(capacity, bool) or not isinstance(capacity, int):
            raise TypeError("capacity must be an integer")
        if capacity <= 0:
            raise ValueError("capacity must be positive")
        self._code = code.strip()
        self._capacity = capacity
        self._students: dict[str, Student] = {}
        self._marks: dict[str, float] = {}
```

Read the class docstring as the practice contract. It states the duplicate-ID rule, capacity rejection and mark range before we design tests. A test should derive its expectation from these rules rather than from whatever the code happens to return.

The dictionaries hold students by ID and marks by ID. They are implementation details. Tests will use public properties and methods to prepare and observe the model instead of editing those dictionaries directly.

Keep the four-space indentation inside Unit. The next two sections continue this same class; do not write another `class Unit:` heading or repeat its constructor.

5. Application code, part 3: Public observations

Append this next part to `enrolment.py`. Every decorator and method below remains indented inside the Unit class. A property allows a caller to read `unit.remaining_places` rather than call it as a method.

```
    @property
    def code(self) -> str:
        return self._code

    @property
    def capacity(self) -> int:
        return self._capacity

    @property
    def students(self) -> tuple[Student, ...]:
        """Immutable roster snapshot in enrolment order."""
        return tuple(self._students.values())

    @property
    def marks(self) -> dict[str, float]:
        """Independent dictionary snapshot; editing it cannot edit the unit."""
        return dict(self._marks)

    @property
    def remaining_places(self) -> int:
        return self.capacity - len(self._students)

    @property
    def is_full(self) -> bool:
        return self.remaining_places == 0
```

The roster is returned as a tuple snapshot in enrolment order. The marks property returns a separate dictionary, so a caller cannot change the unit's marks by editing that returned dictionary. These are public promises that the later snapshot tests verify.

`remaining_places` is derived from capacity and roster size. `is_full` is derived from remaining places. An exact count assertion proves a different observation from a boolean fullness assertion; choose the one that matches the case.

6. Application code, part 4: Actions and display

Append this final part to `enrolment.py`. The first five methods continue inside Unit. At the end, `display_unit` returns to the left margin: it is a separate function that uses a Unit object. The four application parts together form the complete file.

```
    def enrol(self, student: Student) -> bool:
        if not isinstance(student, Student):
            raise TypeError("student must be a Student")
        if student.student_id in self._students:
            return False
        if self.is_full:
            raise ValueError("Unit is full")
        self._students[student.student_id] = student
        return True

    def find_student(self, student_id: str) -> Student | None:
        return self._students.get(student_id)

    def record_mark(self, student_id: str, mark: float) -> None:
        if student_id not in self._students:
            raise LookupError("Student is not enrolled")
        if isinstance(mark, bool) or not isinstance(mark, (int, float)):
            raise TypeError("mark must be a number")
        if not 0 <= mark <= 100 or not isfinite(mark):
            raise ValueError("mark must be finite and between 0 and 100")
        self._marks[student_id] = float(mark)

    def average_mark(self) -> float | None:
        """Mean of recorded marks only; None when there are no marks."""
        if not self._marks:
            return None
        return sum(self._marks.values()) / len(self._marks)

    def summary(self) -> str:
        return f"{self.code}: {len(self.students)}/{self.capacity} enrolled"


def display_unit(unit: Unit) -> None:
    """Small presentation boundary for an optional captured-output example."""
    print(unit.summary())
```

Rejection checks happen before mutation. The mean includes recorded marks only. An absent mean is None, which is different from a mean of zero. The display function prints the summary with a newline. We can now design tests for these observations.

7. Write your first proper test file

Before writing Python, state the case: a new unit has three places; one new student enrols; two places should remain. The expected two follows from the contract and arithmetic, not from copying the application's observed result.

Test role	This case
Arrange	A fresh three-place Unit and a fixed synthetic Student.
Act	Enrol that student once.
Assert	The observable remaining_places value equals two.
Limit	Does not establish duplicate handling, full-unit rejection or marking.

Create `tests/test_assertions.py` with this complete first version:

```
from enrolment import Student, Unit


def test_enrolment_reduces_remaining_places():
    unit = Unit("TECH101", capacity=3)       # Arrange
    student = Student("S101", "Alex Example")
    unit.enrol(student)                      # Act
    assert unit.remaining_places == 2        # Assert
```

The file and function names begin with `test_`, allowing pytest to discover the case. The import comes from the application module. Test functions normally return no value: the assertions decide whether their observations meet the expectation. There is no need to call the test manually or add a main block.

Save both files and run the test file from `unit_enrolment_practice/`:

```
python -m pytest tests/test_assertions.py -v
```

The verified result is one collected test and **1 passed**. If an import or collection error appears, check the folder, interpreter, filenames and indentation first; the intended behavioural test may not have run.

This first version uses clear local setup. The next section introduces optional fixtures and explains how to replace this short file with the larger catalogue. Do not append a second definition with the same test name: Python would replace the earlier definition rather than preserve two cases.

8. Assemble a proper test module with fresh setup

After the first-test exercise, replace that short version of `tests/test_assertions.py`. Begin the replacement file with these imports, then append the test functions from all twelve groups in sections 9–20, in order. Do not copy explanatory prose, section headings or code-fence markers. Every function belongs in this one test file.

```
import pytest

from enrolment import Student, Unit, display_unit
```

Create a separate `tests/conftest.py` file with this complete code. pytest automatically discovers it:

```
import pytest

from enrolment import Student, Unit

@pytest.fixture
def student():
    return Student("S101", "Alex Example")

@pytest.fixture
def unit():
    return Unit("TECH101", capacity=3)
```

Each requesting test receives fresh objects. A test parameter named `unit` asks pytest for that fixture; the test does not call it manually. Tests share one model but do not share mutable state. Fixture scope defaults to one test function.

Arrange establishes a starting state; Act performs the behaviour; Assert checks the outcome. For example, enrolment prepares the state for a mark test, while recording the mark is that test's target action. Only place the expected failing operation inside `pytest.raises`.

Keep `enrolment.py` as application code, `conftest.py` as reusable test setup and `test_assertions.py` as the assertions. Each test has a unique `test_` function name. Keep parametrisation decorators immediately above their function and retain indentation. Test discovery uses the `test_*.py` filename convention shown here.

```
python -m pytest tests/test_assertions.py
python -m pytest -k snapshot
```

Once all groups are assembled, the first command reports 51 passes; the second selects four cases and deselects 47. Run the full suite after a focused selection. The count is a verification result, not an assessment target. If fewer cases are collected, check that no group is missing and no function name was accidentally repeated.

9. Equality and inequality

Equality (==) compares values. The first two tests compare a public field and an observable count with independently expected values. Inequality (`!=`) establishes that the two Student value objects differ.

```
def test_code_equals_expected_text(unit):
    assert unit.code == "TECH101"

def test_enrolment_reduces_remaining_places(unit, student):
    unit.enrol(student)
    assert unit.remaining_places == 2

def test_two_different_students_are_not_equal():
    first = Student("S101", "Alex Example")
    second = Student("S102", "Sam Example")
    assert first != second
```

These assertions do not inspect private dictionaries. A count alone cannot establish which student is present; later examples check contents and identity.

10. Booleans and truthiness

Use `is True` or `is False` when the contract promises an actual boolean. Truthiness uses `assert value` or `assert not value`; it checks whether a value behaves as nonempty/empty or true/false.

```
def test_new_enrolment_returns_true(unit, student):
    assert unit.enrol(student) is True

def test_duplicate_returns_false(unit, student):
    unit.enrol(student)
    assert unit.enrol(student) is False
    assert len(unit.students) == 1

def test_roster_is_empty_then_nonempty(unit, student):
    assert not unit.students
    unit.enrol(student)
    assert unit.students
```

A nonempty roster assertion is weaker than checking its exact contents. Likewise `assert result` would accept integer 1, whereas `result is True` checks the boolean singleton. The duplicate case also checks that no extra place was occupied.

11. Identity and optional results

Use `is` to check the same object and `is not` to check distinct objects. Student equality is value equality because it is a dataclass; two separately constructed equal values are still distinct objects.

```
def test_lookup_retains_the_same_student_object(unit, student):
    unit.enrol(student)
    assert unit.find_student("S101") is student

def test_equal_students_need_not_be_the_same_object():
    first = Student("S101", "Alex Example")
    second = Student("S101", "Alex Example")
    assert first == second
    assert first is not second

def test_missing_lookup_is_none(unit):
    assert unit.find_student("S999") is None
```

```
def test_existing_lookup_is_not_none(unit, student):
    unit.enrol(student)
    assert unit.find_student("S101") is not None
    # Existence alone is weaker than the identity check above.
```

Use `is None` and `is not None` for an optional result. Existence alone does not establish that lookup found the intended student. Do not use identity to compare ordinary numbers or strings; use equality for their values.

12. Membership, sequence contents and size

`in` and `not in` check membership. Membership in this roster uses Student equality, not object identity. Tuple equality checks exact contents and order; the model explicitly promises enrolment order.

```
def test_roster_contains_enrolled_student(unit, student):
    unit.enrol(student)
    assert student in unit.students
    assert Student("S999", "Taylor Example") not in unit.students

def test_roster_preserves_enrolment_order(unit, student):
    second = Student("S102", "Sam Example")
    unit.enrol(student)
    unit.enrol(second)
    assert unit.students == (student, second)

def test_roster_has_exact_size(unit, student):
    unit.enrol(student)
    assert len(unit.students) == 1

def test_student_id_set_ignores_order(unit, student):
    unit.enrol(Student("S102", "Sam Example"))
    unit.enrol(student)
    assert {s.student_id for s in unit.students} == {"S101", "S102"}
```

Length checks size only. A set comparison checks the unique IDs irrespective of order, but discards multiplicity: duplicate elements collapse into one set member. Select the observation that matches the claim.

13. Mappings and types

Dictionary membership checks keys, and indexing observes one value. Whole-dictionary equality checks all key/value entries, irrespective of insertion order. A single value assertion could miss an unexpected extra key.

```
def test_mark_snapshot_has_expected_key_and_value(unit, student):
    unit.enrol(student)
    unit.record_mark("S101", 72)
    assert "S101" in unit.marks
    assert "S999" not in unit.marks
    assert unit.marks["S101"] == 72.0

def test_mark_snapshot_equals_entire_expected_mapping(unit, student):
    unit.enrol(student)
    unit.record_mark("S101", 72)
    assert unit.marks == {"S101": 72.0}

def test_public_results_have_documented_types(unit, student):
    unit.enrol(student)
    assert isinstance(unit.find_student("S101"), Student)
    assert isinstance(unit.students, tuple)
    assert isinstance(unit.marks, dict)
```

```
def test_enrol_returns_an_actual_boolean(unit, student):
    result = unit.enrol(student)
    assert type(result) is bool
    # Exact type matters here because the contract promises bool, not 1.
```

`isinstance` accepts instances of a class and its subclasses. `type(result) is bool` is an exact-type check, justified here by an explicit bool return contract. Type alone does not establish correct values.

14. Numerical comparisons and boundaries

Numeric comparisons express bounds and changes. `0 <= remaining <= capacity` is a chained inclusive range assertion; < and > provide strict comparisons. A range check is weaker than the exact expected count.

```
def test_remaining_places_stay_in_range(unit, student):
    unit.enrol(student)
    assert 0 <= unit.remaining_places <= unit.capacity
    assert unit.remaining_places < unit.capacity

def test_last_available_place_can_be_taken(student):
    small_unit = Unit("TECH101", 1)
    assert small_unit.remaining_places > 0
    small_unit.enrol(student)
    assert small_unit.remaining_places == 0
    assert small_unit.is_full is True

@pytest.mark.parametrize("mark", [0, 100])
def test_mark_accepts_both_inclusive_boundaries(unit, student, mark):
    unit.enrol(student)
    unit.record_mark("S101", mark)
    assert unit.marks["S101"] == mark
```

The last-place case checks the capacity boundary. Parametrisation collects one mark case for 0 and one for 100, the explicitly included endpoints. Boundaries come from the practice contract, not from an invented restriction.

15. Floating-point comparisons and ungraded results

Floating-point arithmetic may not represent decimal fractions exactly. Use `pytest.approx` when a numerical tolerance is justified; the first case uses an explicit absolute tolerance of 1e-12 around 0.15.

```
def test_mean_uses_an_explicit_float_tolerance(unit, student):
    unit.enrol(student)
    unit.enrol(Student("S102", "Sam Example"))
    unit.record_mark("S101", 0.1)
    unit.record_mark("S102", 0.2)
    assert unit.average_mark() == pytest.approx(0.15, abs=1e-12)

def test_no_recorded_marks_mean_none(unit, student):
    unit.enrol(student)
    assert unit.average_mark() is None

def test_mean_excludes_ungraded_students(unit, student):
    unit.enrol(student)
    unit.enrol(Student("S102", "Sam Example"))
    unit.record_mark("S101", 80)
    assert unit.average_mark() == pytest.approx(80.0)
```

Do not round away a defect or choose a tolerance merely to accept the observed result. `is None` distinguishes an absent average from a zero mark. The third case checks that the denominator excludes ungraded students.

16. Strings and optional captured output

Exact string equality checks the entire summary. Prefix, suffix and substring checks demonstrate partial observations; they can all pass while other text is wrong. Use them when the contract promises only that part.

```
def test_summary_matches_exact_text(unit, student):
    unit.enrol(student)
    assert unit.summary() == "TECH101: 1/3 enrolled"

def test_summary_prefix_suffix_and_substring(unit, student):
    unit.enrol(student)
    summary = unit.summary()
    assert summary.startswith("TECH101:")
    assert summary.endswith("enrolled")
    assert "1/3" in summary
    # These partial checks do not replace an exact-text contract.

def test_display_writes_summary_and_no_error(unit, capsys):
    display_unit(unit)
    captured = capsys.readouterr()
    assert captured.out == "TECH101: 0/3 enrolled\n"
    assert captured.err == ""
```

The optional built-in `capsys` fixture captures output. `readouterr()` supplies stdout and stderr; the newline is part of the display contract. No additional capture plugin is needed.

17. Exceptions, messages and exact exception type

`pytest.raises` is an expectation about rejection, not a Python assert statement. It accepts the named exception and its subclasses. `match` uses a regular expression; anchors here require the entire promised message.

```
def test_unknown_student_cannot_receive_a_mark(unit):
    with pytest.raises(LookupError):
        unit.record_mark("S999", 70)

def test_full_unit_reports_the_documented_message(student):
    small_unit = Unit("TECH101", 1)
    small_unit.enrol(student)
    with pytest.raises(ValueError, match=r"^Unit is full$"):
        small_unit.enrol(Student("S102", "Sam Example"))

def test_blank_id_has_exact_error_type_and_message():
    with pytest.raises(ValueError) as error:
        Student("   ", "Alex Example")
    assert error.type is ValueError
    assert str(error.value) == "student_id must not be blank"

@pytest.mark.parametrize("mark", [
    -0.1, 100.1, float("nan"), float("inf"), 10**400,
])
def test_invalid_numeric_marks_are_rejected(unit, student, mark):
    unit.enrol(student)
    with pytest.raises(ValueError):
        unit.record_mark("S101", mark)
```

The exception-info assertions demonstrate exact exception type and message. Exact wording is appropriate only because this teaching model fixes those messages. The numeric rejection cases cover just-outside boundaries and non-finite marks.

18. Unchanged state and independent snapshots

A raised exception does not prove that data remained unchanged. Compare public state with a separate pre-action snapshot after rejection. The existing-mark assertion ensures invalid input did not silently replace the earlier mark.

```
def test_rejected_mark_preserves_existing_mark(unit, student):
    unit.enrol(student)
    unit.record_mark("S101", 70)
    before = unit.marks
    with pytest.raises(ValueError):
        unit.record_mark("S101", 101)
    assert unit.marks == before
    assert unit.marks == {"S101": 70.0}

def test_editing_mark_snapshot_cannot_change_unit(unit, student):
    unit.enrol(student)
    unit.record_mark("S101", 70)
    snapshot = unit.marks
    other_snapshot = unit.marks
    assert snapshot == other_snapshot
    assert snapshot is not other_snapshot
    snapshot["S101"] = 0
    assert unit.marks == {"S101": 70.0}

def test_roster_snapshot_does_not_grow_later(unit, student):
    unit.enrol(student)
    before = unit.students
    unit.enrol(Student("S102", "Sam Example"))
    assert before == (student,)
    assert len(unit.students) == 2

def test_rejected_enrolment_preserves_roster_and_capacity(student):
    small_unit = Unit("TECH101", 1)
    small_unit.enrol(student)
    before = small_unit.students
    with pytest.raises(ValueError):
        small_unit.enrol(Student("S102", "Sam Example"))
    assert small_unit.students == before
    assert small_unit.remaining_places == 0
```

Snapshot equality checks contents; `is not` checks distinct containers. Editing the returned dictionary must not edit the Unit. The tuple snapshot retains its old contents after later enrolment. None of these tests reads or writes private dictionaries.

19. All, any and diagnostic messages

`all` checks every member of an iterable; `any` checks whether at least one meets a predicate. `all([])` is True, so establish a nonempty roster when the claim requires actual members. `any([])` is False.

```
def test_all_roster_items_are_students(unit, student):
    unit.enrol(student)
    unit.enrol(Student("S102", "Sam Example"))
    assert len(unit.students) == 2
    assert all(isinstance(s, Student) for s in unit.students)
    # all([]) is True: size makes the nonempty claim explicit.

def test_at_least_one_roster_item_has_target_id(unit, student):
    unit.enrol(student)
    assert any(s.student_id == "S101" for s in unit.students)
```

```
def test_count_assertion_has_a_useful_failure_message(unit, student):
    unit.enrol(student)
    assert len(unit.students) == 1, "One new ID should occupy one place"
```

A diagnostic message after the comma supplements pytest failure output. It does not make the assertion stronger. Describe the promised observation, rather than using a message such as "test failed".

20. Parametrised contracts and independent units

Parametrisation runs the same rejection contract over multiple inputs. Each collected case retains its own outcome; more data does not automatically mean more behavioural coverage. Booleans are deliberately excluded from marks even though bool is a Python int subclass.

```
@pytest.mark.parametrize("capacity", [0, -1, -5])
def test_capacity_must_be_positive(capacity):
    with pytest.raises(ValueError):
        Unit("TECH101", capacity)

@pytest.mark.parametrize("mark", [True, "70", None])
def test_non_numeric_or_boolean_mark_is_rejected(unit, student, mark):
    unit.enrol(student)
    with pytest.raises(TypeError):
        unit.record_mark("S101", mark)

def test_distinct_units_have_independent_rosters(student):
    first = Unit("TECH101", 3)
    second = Unit("TECH102", 3)
    first.enrol(student)
    assert first is not second
    assert first.students == (student,)
    assert second.students == ()

def test_duplicate_id_keeps_original_object_even_when_full(student):
    small_unit = Unit("TECH101", 1)
    small_unit.enrol(student)
    duplicate = Student("S101", "Different Example")
    assert small_unit.enrol(duplicate) is False
    assert small_unit.find_student("S101") is student
    assert small_unit.remaining_places == 0
```

The final cases check independent Unit instances and the documented duplicate-ID precedence at capacity. A duplicate keeps the original Student rather than overwriting it. These are explicit contracts of this independent teaching project.

21. Choosing an assertion and investigating a failure

Form	Use when the claim is about
== / !=	Equal/different values
is / is not	Object identity or the True/False/None singletons
assert value / assert not value	Truthiness, including nonempty/empty collections
in / not in	Membership; keys for dictionaries
tuple / set / dict equality	Exact ordered contents / unique membership / all key-values
len(result) == n	Exact collection size
isinstance / type(...) is	Type compatibility / explicitly promised exact type
<, >, <=, >=, chained range	Numerical boundaries and relationships
pytest.approx	A numerical result within a justified tolerance
startswith / endswith / substring	A promised part of text
pytest.raises	A documented rejection; check state separately if promised
all / any	Every member / at least one member meets a condition

Run one floating-point example:

```
python -m pytest tests/test_assertions.py::test_mean_uses_an_explicit_float_tolerance
```

On a disposable copy, change the expected remaining places in the second equality test from 2 to 3. Run it:

```
python -m pytest tests/test_assertions.py::test_enrolment_reduces_remaining_places
```

The observed abbreviated failure is `assert 2 == 3`. The model is correct: one enrolment consumed one of three places. Restore the expectation to 2 and rerun the full suite. This is a test-expectation error, not a reason to change the implementation.

Avoid `assert True`, copying the actual result into the expected result, broad exception catching, unnecessary private-state assertions and combining unrelated behaviour in one test. A passing partial assertion may still miss a relevant wrong result.

Further reference: official pytest 8.3 assertion and exception guidance and approx API. The project uses pinned pytest 8.3.5.